

In deze reflectie toon ik aan hoeveel werk en persoonlijke inzet ik geleverd heb in de verdere ontwikkeling van mijn educatieve game 'Save The Planet'.

Alle onderstaande toevoegingen en verbeteringen zijn specifiek uitgewerkt in het kader van mijn **tweede zit**.

Na de eerste feedbackronde (8/20 wegens "onvoldoende eigen inzet") heb ik besloten om mijn bijdrage grondig uit te breiden. In totaal heb ik **18 onderdelen volledig zelf ontworpen, gecodeerd of geanalyseerd**, inclusief animaties, interactieve systemen, visuele elementen, en gameplay-mechanica.

Elk onderdeel wordt hieronder concreet en gedetailleerd toegelicht, met uitleg over het technische proces en de pedagogische keuzes.

1.

In deze versie heb ik een dynamisch weersysteem geïmplementeerd dat reageert op de voortgang van de missie in het bos.

- Ik heb twee Particle Systems gecreëerd: **RainHeavy** (zware regen) en **RainLight** (lichte regen).
- Met behulp van een op maat gemaakte **WeatherManager**-script verandert het weer automatisch afhankelijk van de status van de "Bomenplantmissie".
- De overgang tussen weersituaties gebeurt **geleidelijk**, dankzij een fade-in/fade-out effect via code (geen plotselinge visuele switch).
- Het gedrag is als volgt:
 - **Voor de missie:** zware regen.
 - **Tijdens de missie:** lichte regen.
 - **Na voltooiing van de missie:** geen regen meer.

Dit weersysteem versterkt de **narratieve en educatieve boodschap** van het spel: het klimaat wordt beter naarmate de speler actie onderneemt voor de planeet.

2.

Optiemenu – Geluidsinstellingen

In mijn spel heb ik een **optiescher**m toegevoegd waarin de speler het volume van de muziek en de geluidseffecten (SFX) kan aanpassen.

Hiervoor heb ik een extra **Canvas** gemaakt dat verschijnt wanneer je op de knop “Options” klikt in het hoofdmenu.

Het optiemenu bevat:

- Een **muziekslider** om het achtergrondvolume aan te passen.
- Een **SFX-slider** om het volume van de geluidseffecten te regelen.
- Een **“Back” knop** om terug te keren naar het hoofdmenu.

Ik heb hiervoor eigen scripts geschreven:

- Het script `AudioSettings.cs` zorgt ervoor dat de sliders dynamisch verbonden zijn met het geluidsniveau van de muziek en SFX. De wijzigingen worden automatisch opgeslagen met `PlayerPrefs`, zodat de instellingen bij het opnieuw opstarten van het spel behouden blijven.
- Voor de **“Back” knop** heb ik een apart script `BackToMainMenu.cs` gemaakt. Dit script activeert het hoofdmenu opnieuw en verbergt het optiescher wanneer je teruggaat.

Daarnaast heb ik een **klikgeluid (SFX)** toegevoegd aan alle knoppen (inclusief in het menu), zodat elke actie een hoorbare feedback geeft. Dit versterkt de gebruikerservaring.

3.

Wind Effect op het Strand met Subtiele Objectbeweging

In deze versie heb ik een visueel wind-effect toegevoegd aan de strandscène van het spel, om de omgeving levendiger en dynamischer te maken.

Ik heb hiervoor een **eigen Particle System** gemaakt dat de wind visueel simuleert. Deze wind beweegt voortdurend, op een rustige en natuurlijke manier.

Om het effect geloofwaardiger te maken, heb ik ook een script toegevoegd aan de plastic flessen op het strand. Dit script zorgt ervoor dat de flessen **lichtjes wiegen en bewegen**, alsof ze worden beïnvloed door de wind.

Het gedrag is als volgt:

De wind is continu zichtbaar als een stroom in de lucht.

De flessen bewegen zachtjes heen en weer, elk met een iets andere timing voor een realistischer effect.

Dankzij deze toevoegingen voelt de strandomgeving meer **levend en geloofwaardig** aan, zonder dat dit de speler afleidt.

Het versterkt de **natuurlijke sfeer van het spel** en draagt bij aan de totale immersie in de spelwereld.

4.

Aangepast karakter en realistische graafanimatie

Voor dit project heb ik ervoor gekozen om mijn eigen speelbare karakter te ontwerpen via een pixel art character generator, die ik vervolgens **volledig aangepast heb aan mijn spel**. Het karakter beschikt over **animaties in vier richtingen** (omhoog, omlaag, links, rechts), en bevat ook een **specifieke graafanimatie met een schop**.

Om deze animaties te beheren, heb ik een **custom animatiesysteem** opgezet via twee scripts:

- **AnimatorCharacter** – voor het aansturen van de juiste animatie op basis van richting en beweging.
- **AnimatorSprite** – voor het afspelen van sprites frame per frame met een constante snelheid, buiten Unity's Animator.

Interactieve graafanimatie bij planten

Bij interactie met een **PlantSpot**, wordt – indien de speler een schop bezit – de graafanimatie automatisch afgespeeld via een coroutine (**PlayDigAnimation**). Deze coroutine roept een **PlayStepByStep()** functie aan die alle sprites van de graafanimatie **volledig en vloeiend** toont met een kleine pauze tussen frames.

Na het voltooien van de animatie gaat het systeem verder met de logica van de missie.

Bugs en oplossingen

1. Animatie toonde enkel het eerste frame

In de eerste versie werd enkel het eerste sprite van de animatie getoond, omdat **Start()** onmiddellijk terugzetten naar de eerste frame.

Oplossing: Implementatie van **PlayStepByStep()** die sprite-per-sprite toont binnen een coroutine.

2. Speler kon bewegen tijdens de animatie

Tijdens de graafanimatie kon de speler nog vrij bewegen, wat visueel verwarrend en foutgevoelig was.

Oplossing:

- Tijdens de coroutine werd `canMove` (in het `CharacterMovement`-script) tijdelijk op `false` gezet.
 - Na de animatie werd deze terug op `true` gezet.
-

3. Snel achter elkaar op spatie drukken (spam) bugde de animatie

Wanneer de speler herhaaldelijk op spatie drukte tijdens het graven, werd de animatie meerdere keren gestart, wat tot visuele bugs en logische conflicten leidde.

Oplossing:

- Gebruik van een boolean `isInteracting` in `PlantSpot`, die tijdelijk op `true` staat tijdens `HandleInteraction()` om dubbele triggers te blokkeren.
-

4. Animatie speelde zelfs af als de speler geen schop had

De animatie werd altijd gestart, ook als de speler geen schop had en de interactie nadien werd geannuleerd.

Oplossing:

- Verplaatst de aanroep van `PlayDigAnimation()` na de check of de speler een schop heeft.
Zo wordt de animatie **alleen afgespeeld als de actie ook effectief doorgaat**.
-

Resultaat

Door al deze optimalisaties werkt de graafanimatie nu op een **realistische, gecontroleerde en foutloze manier**.

Elke interactie is visueel duidelijk, technisch correct en draagt bij aan de immersie van de speler in het spel.

5.

Realistische zaai-animatie bij het planten van bomen

Voor deze feature heb ik volledig zelf een **plant-animatie** toegevoegd aan mijn speelbare karakter om de actie van het **zaaien van zaden** visueel geloofwaardig en educatief te maken. Dit gebeurde in verschillende stappen:

Nieuwe sprites ontworpen voor het zaaien

Ik ben gestart met het zoeken van geschikte sprite-frames die visueel tonen hoe een personage zaden gooit. Hiervoor gebruikte ik een externe pixel-art generator en heb ik specifieke frames gekozen die goed passen bij de rest van mijn spelstijl. Deze sprites werden manueel toegevoegd in Unity in vier richtingen: omhoog, omlaag, links en rechts.

Aanpassing van het Animator-systeem

Om de animatie correct af te spelen, heb ik mijn bestaande `AnimatorCharacter`-systeem uitgebreid. Naast de loop- en graafanimaties heb ik ook:

- Vier nieuwe lijsten toegevoegd: `plantDownSprites`, `plantUpSprites`, `plantLeftSprites`, `plantRightSprites`.
 - Voor elk van deze richtingen heb ik een nieuwe `AnimatorSprite` instantie aangemaakt (vb. `plantDownAnim`).
 - Een nieuwe coroutine `PlayPlantAnimation(WatchingDirection direction)` geschreven, die de frames **één voor één afspeelt** met een korte delay. Zo blijft alles handmatig controleerbaar, buiten Unity's Animator-systeem.
 - Tijdens de animatie wordt `isPlayingSpecialAnimation` op `true` gezet, zodat andere bewegingen tijdelijk worden geblokkeerd.
-

Integratie in het PlantSpot-systeem

Bij interactie met een `PlantSpot`, controleert het systeem of de speler zaden heeft. Als dit het geval is:

1. De speler wordt tijdelijk vergrendeld (`CanMove = false`).
2. De plant-animatie wordt gestart met
`character.Animator.PlayPlantAnimation(direction)`.
3. Pas daarna wordt de eigenlijke boom geplant en de missie voortgezet.

Dit zorgt ervoor dat de zaai-animatie **alleen afspeelt wanneer het logisch is** en dat de actie **niet kan worden gespamd** of onderbroken.

6.

Oprap-animatie bij het verzamelen van plastic flessen

Voor deze feature heb ik zelf een oprap-animatie toegevoegd aan mijn speelbaar personage, zodat het verzamelen van afval op het strand geloofwaardiger en visueel aantrekkelijker wordt. Deze animatie is specifiek bedoeld voor de missie waarbij plastic flessen moeten worden opgeraapt.

Toevoegen van eigen sprites

Ik ben begonnen met het zoeken van een sprite sheet die een subtiele maar duidelijke beweging toont waarbij het personage voor zich grijpt. Deze sprite heb ik manueel in Unity geïmporteerd en opgedeeld in afzonderlijke frames via de Sprite Editor. Omdat in mijn spel het personage meestal naar boven kijkt tijdens deze actie, heb ik enkel de frames voor de "Up"-richting geselecteerd en verwerkt.

Uitbreiding van het AnimatorCharacter-systeem

Binnen mijn eigen animatiesysteem (`AnimatorCharacter.cs`) heb ik:

- Een nieuwe lijst toegevoegd: `switchUpSprites`, waarin ik de frames van de oprap-animatie heb gestoken.
- Een bijhorende `AnimatorSprite` instantie aangemaakt: `switchUpAnim`.
- Een coroutine geschreven: `PlaySwitchAnimationUp()`, die handmatig de frames één voor één afspeelt met een kleine vertraging. Dit laat me toe om volledige controle te behouden over de animatie zonder Unity's standaard Animator-controller te gebruiken.
- De boolean `isPlayingSpecialAnimation` zorgt ervoor dat tijdens het afspelen van deze animatie geen andere bewegingen mogelijk zijn, waardoor het geheel vloeiend en gecontroleerd aanvoelt.

Integratie in het Object-interactiesysteem

In mijn `Object.cs` script (dat instaat voor het oppakken van voorwerpen zoals plastic flessen), heb ik het volgende toegevoegd:

- Eerst controleert het systeem of de missie actief is.
- Als dat het geval is, wordt `PlaySwitchAnimationUp()` uitgevoerd via het character object, op basis van zijn huidige kijkrichting (`ViewDirection`).

- Pas nadat de animatie is afgespeeld, wordt de fles verwijderd uit de scène en het aantal ingezamelde stukken afval verhoogd.

Deze aanpak garandeert dat de animatie alleen afgespeeld wordt wanneer de speler effectief een fles opruimt, wat zorgt voor een meer geloofwaardige en educatieve spelervaring.

7.

Realistische schakel-animatie bij het uitschakelen van lichten (alleen richting omhoog)

Om het uitschakelen van lampen visueel geloofwaardiger te maken en beter aan te laten sluiten bij de spelervaring, heb ik zelf een speciale animatie toegevoegd wanneer de speler een licht uitdoet, **maar enkel als hij naar boven kijkt**.

Toevoegen van eigen sprites

Ik heb een specifieke sprite sheet gekozen met een subtiele beweging die lijkt op het indrukken van een knop of schakelaar. Deze heb ik handmatig gesliced in Unity en enkel de frames voor de **"Up"-richting** gebruikt, omdat de lichten in mijn spel zich meestal boven de speler bevinden.

Uitbreiding van mijn animatiesysteem

In mijn bestaande `AnimatorCharacter`-systeem heb ik:

- Een nieuwe lijst `switchUpSprites` toegevoegd.
- Een aparte `AnimatorSprite` instantie aangemaakt: `switchUpAnim`.
- Een coroutine `PlaySwitchAnimationUp()` geschreven, die de frames van de animatie afspeelt met een korte delay.
- De boolean `isPlayingSpecialAnimation` gebruikt om andere bewegingen tijdelijk te blokkeren tijdens de animatie.

Integratie in het `LightSwitch`-systeem

In het `LightSwitch`-script wordt bij interactie gecontroleerd of de speler naar boven kijkt. Zo ja:

1. De speler wordt tijdelijk vergrendeld.
2. De animatie wordt afgespeeld.
3. Daarna wordt het licht uitgeschakeld, het juiste sprite en de donkere kamer worden geactiveerd.
4. De status van de lamp wordt opgeslagen via `PlayerPrefs`, zodat de toestand behouden blijft na een scènewissel.

Door deze aanpak heb ik een vloeiende, controleerbare en educatief geloofwaardige lichtschakelaarfunctionaliteit gebouwd zonder het standaard Unity Animator-systeem te gebruiken.

8.

DialogBox implementatie

Voor deze stap heb ik een volledig eigen DialogBox geïmplementeerd. In plaats van de standaard UI-componenten van Unity te gebruiken, koos ik ervoor om een eigen visuele stijl te creëren die beter past bij de sfeer van mijn game. Ik heb een eenvoudige rechthoekige sprite gebruikt met een zachte rand, die ik zelf heb toegevoegd en aangepast in Unity.

Daarna heb ik deze sprite gekoppeld aan mijn bestaande dialoogsysteem. De positie, schaal en integratie met de tekstweergave zijn manueel ingesteld zodat alles correct wordt weergegeven tijdens gesprekken. Dit maakt het geheel visueel consistent en geeft de indruk van een old-school RPG dialoogstijl, wat perfect past bij de rest van mijn project.

9.

Unieke NPC-creatie: elk personage zelf ontworpen

Voor mijn project heb ik ervoor gekozen om **alle NPC's (niet-speelbare personages)** volledig zelf te ontwerpen om een unieke visuele identiteit aan mijn game te geven. In plaats van bestaande prefab-characters of kant-en-klare templates te gebruiken, heb ik elk personage **stuk voor stuk gepersonaliseerd** via een karaktergenerator die ik heb gebruikt **als hulpmiddel, niet als eindproduct**.

Met deze tool kon ik een basisvorm maken, maar **ik heb elk detail – zoals de outfits, kapsels, gezichtskenmerken, kleurencombinaties en accessoires – handmatig aangepast en gekozen**. Zo kreeg elk personage een **eigen stijl, persoonlijkheid en functie** binnen het spelverhaal.

Deze aanpak zorgde ervoor dat de NPC's **niet generiek zijn**, maar perfect aansluiten bij het thema van mijn game ("Save The Planet"). Elk personage heeft een betekenisvolle rol: een dorpsbewoner die weetjes deelt, een visser die een missie geeft, een boswachter die bomen wil laten planten... Elk van deze NPC's heeft een eigen look die bij hun omgeving en taak past.

Creatieve meerwaarde

Dankzij dit ontwerpwerk heb ik ook een beter inzicht gekregen in visuele consistentie en thematische opbouw. Alle NPC's passen bij de setting van hun omgeving en bij de boodschap die ik wilde overbrengen. Dit ontwerp is dus niet zomaar grafisch werk, maar een bewuste keuze om **de immersie en de educatieve impact van mijn spel te versterken**.

10.

Eigen ontworpen pixelart-bord als achtergrond voor de quiz

Voor de quiz-scène in mijn spel heb ik ervoor gekozen om een **eigen pixelart-achtergrond** te maken in de vorm van een **groen schoolbord**, volledig in lijn met de educatieve stijl van het project. In plaats van een standaardachtergrond te gebruiken of een kant-en-klaar element uit een asset store te nemen, heb ik dit bord zelf ontworpen in een eenvoudige, duidelijke stijl.

Waarom een bord?

Het idee achter deze achtergrond was om een visuele link te leggen met de **klassieke klasomgeving**, die herkenbaar is voor jongeren van 11 tot 15 jaar – mijn doelpubliek. Zo voelen de quizvragen meer aan als een speelse toets in een klas, wat past bij het educatieve karakter van mijn spel.

Eigen creatie

Ik heb de achtergrond **zelf in pixelstijl opgebouwd**, met eenvoudige kleuren en strakke lijnen, zodat het ontwerp duidelijk en niet afleidend is tijdens het beantwoorden van de vragen. Door dit element zelf te maken toon ik aan dat ik ook oog heb voor **grafische consistentie en thematische details**.

Resultaat: visueel en inhoudelijk coherent

Het eindresultaat is een quiz-omgeving die volledig past binnen de stijl van mijn spel: educatief, vriendelijk en consistent. Deze zelfgemaakte achtergrond versterkt het gevoel van een eigen creatie, wat voor mij belangrijk was na eerdere feedback over eigen inbreng.

11.

AI-gegenereerde achtergrond voor het hoofdmenu met persoonlijke artistieke input

Voor het hoofdmenu van mijn spel heb ik gebruikgemaakt van een **AI-generator om een achtergrondaafbeelding te creëren** die meteen de toon zet voor het avontuur. Hoewel de afbeelding technisch gegenereerd werd, heb ik de stijl, sfeer en compositie volledig bepaald door middel van een **zorgvuldig gekozen persoonlijke prompt**, die volledig aansluit bij de inhoud en de esthetiek van mijn spel.

Mijn creatieve richting

In de prompt heb ik elementen verwerkt die mijn spel typeren: een natuurlijke omgeving, een rustige sfeer, een hoopvolle toon en kleuren die passen bij het ecologische thema van het spel. Het resultaat is geen willekeurige afbeelding, maar een **visueel startscherm dat thematisch en emotioneel in lijn ligt met het verhaal van Save The Planet**.

Waarom AI gebruiken?

Aangezien ik beperkte tijd en middelen had om alles zelf pixel per pixel te tekenen, heb ik ervoor gekozen om dit specifieke element via AI te genereren, **maar wel volledig in functie van mijn visie**. Deze werkwijze laat toe om snel tot een kwalitatief en gepast resultaat te komen, zonder afbreuk te doen aan de originaliteit van het project.

Eigen inbreng en samenhang

Ik heb het resultaat geëvalueerd, aangepast aan mijn user interface, en geïntegreerd in de flow van mijn spel. Zo blijft het eindresultaat **een bewuste en persoonlijke keuze**, niet zomaar een overgenomen beeld. Het hoofdmenu weerspiegelt dus mijn artistieke keuzes en versterkt de herkenbaarheid van mijn game vanaf het eerste moment.

12. Poging tot het implementeren van een richtingspijl (pointer)

In het kader van de ontwikkeling van mijn educatieve game *Save The Planet* heb ik geëxperimenteerd met het toevoegen van een **richtingspijl (pointer)** om de speler naar specifieke doelwitten te leiden, zoals teleportatiepunten (TP's) of missiegebieden.

Creatie van de assets

Om grafisch in lijn te blijven met de stijl van het spel, heb ik zelf:

- een **pijl** in **pixelart** ontworpen, standaard naar rechts gericht,
- en een UI-component genaamd **Pointer** toegevoegd binnen een **Screen Space - Overlay** canvas, zodat deze altijd zichtbaar blijft.

Technische implementatie

Het idee was als volgt:

- De pijl blijft onderaan het scherm staan, onafhankelijk van de positie van het personage.
- In elke frame (**Update()**), draait de pijl automatisch in de richting van een doelwit in de spelwereld (zoals een TP), met behulp van een berekening op basis van de posities van de speler en het doel.
- De draairichting werd bepaald met behulp van **Mathf.Atan2()** om een correcte rotatiehoek te verkrijgen.

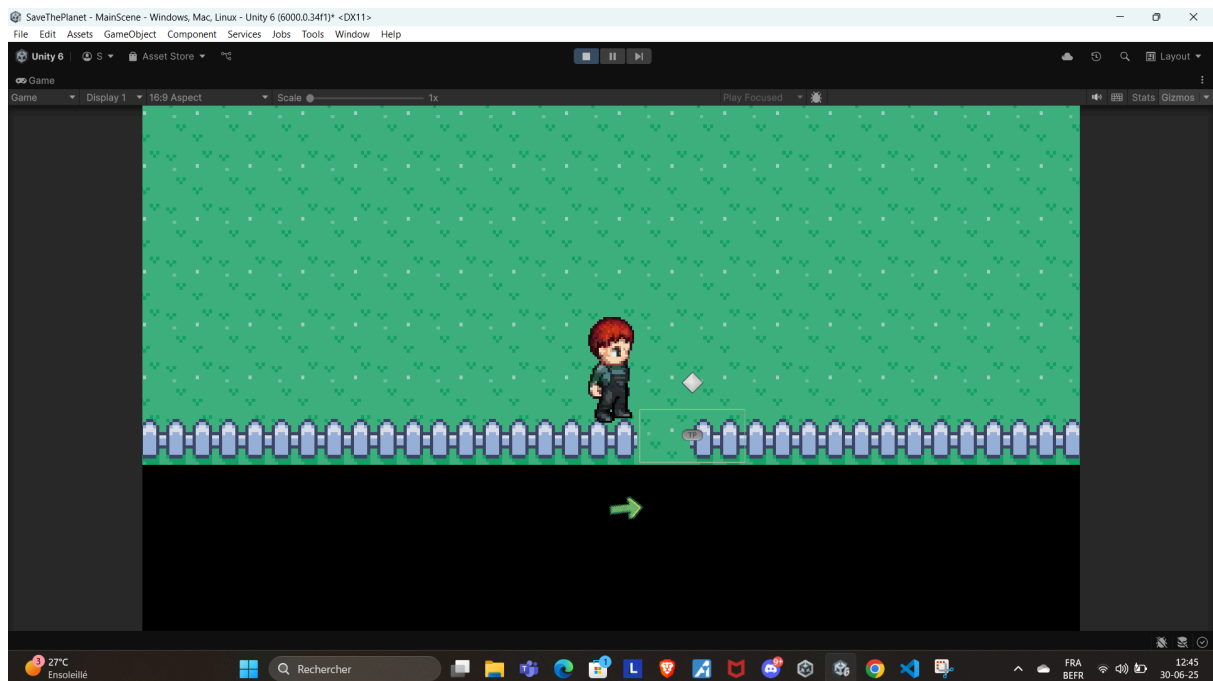
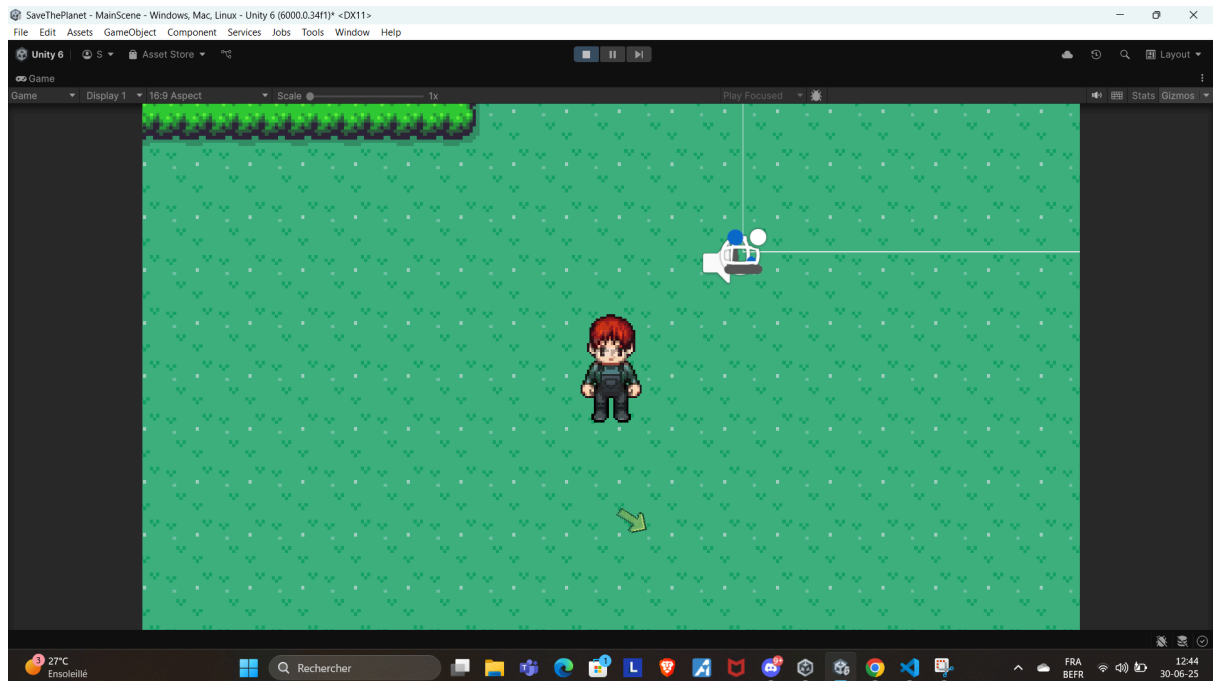
Resultaat en kritische analyse

Hoewel de techniek goed werkte, bleek het eindresultaat **geen echte meerwaarde te bieden voor de gameplay**:

- Omdat de pijl **vast op het scherm** staat, gaf hij onvoldoende duidelijke informatie over **de exacte locatie** waar de speler heen moest.
- Dit maakte de ervaring eerder abstract dan nuttig, zeker in een 2D-omgeving waarin de speler de doelwitten vaak al op het scherm ziet.
- Voor een jonge doelgroep (11–15 jaar) bleek dit systeem minder geschikt dan meer visueel concrete aanwijzingen.

Conclusie

Ik heb daarom besloten **dit systeem niet in de finale versie van het spel op te nemen**. Deze keuze is het resultaat van een **bewuste game design beslissing**: ondanks de succesvolle implementatie was de pointer niet nuttig genoeg voor de spelervaring of de pedagogische doelstelling van het project.



13. Trashcan animatie

Om de speler in staat te stellen afval weg te gooien in de vuilnisbak, heb ik zelf een nieuwe animatie toegevoegd aan het bestaande animatiesysteem.

Ik heb eerst een nieuwe functie `PlayThrowAnimation()` geschreven in mijn `AnimatorCharacter` script, vergelijkbaar met de pickup-animatie die ik eerder had gemaakt.

Vervolgens heb ik aparte sprite-lijsten aangemaakt voor elke richting (boven, onder, links, rechts), zodat de animatie overeenkomt met de kijkrichting van het personage.

In het `TrashCan` script heb ik ervoor gezorgd dat wanneer de speler interactie heeft met de vuilnisbak en afval in zijn inventory heeft, de juiste animatie afgespeeld wordt voordat de dialoog begint.

Deze animatie wordt aangeroepen via een coroutine, net als bij het oppakken van objecten.

Het hele systeem is stap voor stap door mij zelf opgebouwd en geïntegreerd binnen mijn bestaande structuur, met aandacht voor herbruikbaarheid en consistentie met de andere spelmechanieken.

14. Statistieken bovenaan het scherm – mijn eigen indicatorensysteem

Om mijn persoonlijke bijdrage aan het spel zichtbaar te maken en de impact van de speler tastbaar te maken, heb ik zelf een systeem geïmplementeerd dat drie belangrijke statistieken bijhoudt en weergeeft:

Elektriciteitsverspilling, CO₂-vervuiling en Plasticvervuiling

Mijn doel met dit systeem was om niet enkel visuele of technische feedback te geven, maar ook educatieve bewustwording te creëren: elke actie van de speler heeft een concreet gevolg in de spelwereld.

Technische aanpak Ik heb een UI-canvas aangemaakt waarin drie tekstvelden dynamisch aangepast worden via een script dat gekoppeld is aan de acties van de speler:

- Bij het uitdoen van een licht daalt de elektriciteitsverspilling.
- Bij het planten van een boom daalt de CO₂-vervuiling.
- Bij het oprapen van plastic flessen daalt de plasticvervuiling.

Deze waarden starten elk op 100 en dalen in realtime naargelang de speler positieve acties uitvoert. Ik heb `PlayerPrefs` gebruikt om deze waarden persistent te houden tussen scènes.

Lay-out en visuele keuze Ik heb de statistieken linksboven in het scherm geplaatst, in een eenvoudige box met licht transparante achtergrond. De stijl past bij de pixelart-look van de

game en stoort niet het zicht van de speler. De waarden worden live aangepast, wat zorgt voor onmiddellijke feedback.

Educatieve meerwaarde Dit systeem maakt het mogelijk om op het einde van het spel ook een soort eco-score te tonen, gebaseerd op de mate waarin de speler de planeet heeft geholpen. Dit draagt rechtstreeks bij aan de leerdoelstelling van het project: jongeren inzicht geven in milieuproblematieken én de kracht van individuele actie.

Door dit systeem helemaal zelf te bedenken, ontwerpen en implementeren, toon ik aan dat ik actief heb nagedacht over mijn eigen bijdrage en de pedagogische diepgang van mijn game verder heb ontwikkeld.

Om dit systeem meteen begrijpelijk te maken voor de speler, heb ik in de introductiescène een extra tekst toegevoegd die uitlegt dat de statistieken linksboven in het scherm live tonen hoe jij je steentje bijdraagt aan een betere planeet.

15. Nieuwe huizen met unieke stijl en natuurlijke integratie

Om het visuele aspect van mijn spel verder te personaliseren en afstand te nemen van de standaard look van het originele tileset of tutorialmateriaal, heb ik alle huizen vervangen door nieuwe varianten met unieke kleuren. Hierdoor ziet elk gebouw er anders uit, wat de wereld minder repetitief en meer geloofwaardig maakt.

De school en enkele andere gebouwen zijn opgebouwd uit **solid objects** met een eigen pixelstijl (16px, point no filter, multiple), zodat ze volledig aansluiten bij de retro look van mijn game. Bovendien heb ik de ingangen van de huizen in een **foreground-laag** geplaatst. Dit zorgt ervoor dat wanneer de speler een huis binnenstapt, zijn sprite visueel achter de deur verdwijnt. Deze aanpak geeft een natuurlijker gevoel van overgang en versterkt de immersie voor de speler.

16. Volg-mechanisme voor de EcoDog + interactief gedrag + persistente prefab tussen scènes

In deze versie heb ik een nieuwe NPC toegevoegd: de **EcoDog**, een hond die de speler volgt doorheen het spel en symbool staat voor ecologische hulp en vriendschap. Deze toevoeging was technisch uitdagend, omdat het systeem **realistisch moest reageren**, **obstakels moest vermijden** en **scène-overgangen correct moest overleven**.

Technische doelstelling

Mijn doel was om de hond levendig te maken met **realistisch gedrag**, zoals:

- De speler volgen met natuurlijke bewegingen (eerst X, dan Y – nooit diagonaal),
- Obstakels ontwijken (dus niet door muren lopen),
- **Na 15 seconden inactiviteit gaan zitten**,
- En **niet verdwijnen of gedupliceerd worden** bij het wisselen van scènes.

Implementatie en obstakels

Ik heb een script geschreven waarin:

- De hond de afstand tot de speler controleert,
- Beweegt in logische stappen via een coroutine,
- `BoxCollider2D` en `Rigidbody2D` correct gebruikt worden voor botsing met muren.

Ik ben onderweg op enkele problemen gebotst:

- De hond liep in het begin **diagonaal**,
- Hij **negeerde obstakels** door een foutieve collider,
- Hij **verdween of werd verdubbeld** bij scènewissels,
- En hij **beweegde constant**, zelfs wanneer de speler stilstond.

Oplossingen en extra gedrag

1. **Beweging gefixt**

Beweging per as (X of Y) werd correct geïmplementeerd, en `isMoving` werd geblokkeerd bij obstakels.

2. **Collider probleem opgelost**

De foutmelding “collider did not create any collision shapes” werd opgelost door een correcte `BoxCollider2D` in te stellen met een geldige grootte.

3. **Persistentie tussen scènes**

De `EcoDog` werd omgezet naar een prefab binnen mijn `ReusableObjects` systeem. Met `DontDestroyOnLoad()` en een spawncheck blijft de hond uniek én persistent tussen scènes.

4. **Gedrag bij inactiviteit**

Als de speler 15 seconden niet beweegt, detecteert de hond dit via een timer en **gaat hij automatisch zitten**.
Zodra de speler terug beweegt, staat hij weer recht.

Pedagogische meerwaarde

De `EcoDog` is niet zomaar een NPC, maar een **symbolisch figuur** die staat voor ecologische trouw, loyaliteit en verbondenheid.

Zijn gedrag — volgen, reageren, wachten, kwispelen — maakt hem herkenbaar en geliefd voor jonge spelers.

Resultaat

Het eindresultaat is een **levendige, meebewegende en interactieve hond** die:

- Obstakels vermijdt,
- Zich menselijk gedraagt bij inactiviteit,
- Emotioneel reageert op objecten,
- En **over scènes heen blijft bestaan zonder bugs**.

Deze feature toont mijn **technische creativiteit, debug-vaardigheden en aandacht voor detail**, en draagt bij aan de algemene kwaliteit en emotionele beleving van het spel.

17. Eigen ontworpen tiles voor strand, water, bos en gras – visuele originaliteit en coherentie

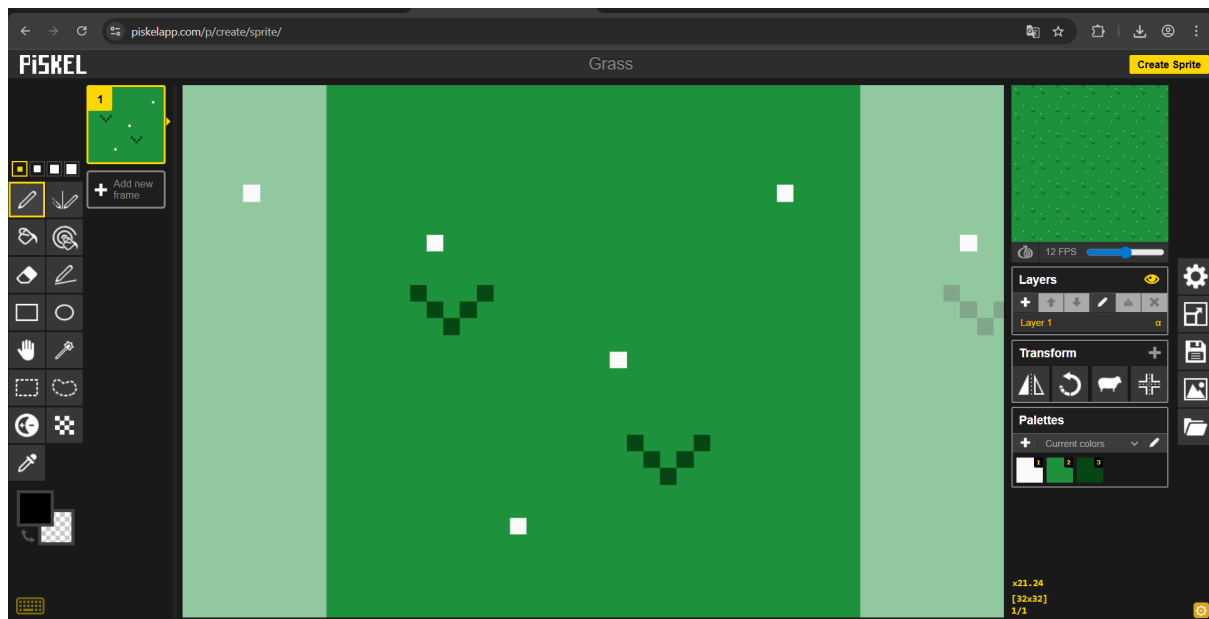
Om de visuele eenheid van mijn spel te versterken en mijn persoonlijke bijdrage verder te bewijzen, heb ik zelf eenvoudige tiles ontworpen voor vier fundamentele elementen van de spelwereld: **strand, water, bosgrond, gras** en een **eigen vloertileset voor de interieurs**.

Deze tiles heb ik **zelf pixel per pixel getekend**, met een beperkte kleurensset en in een eenvoudige stijl die past bij de rest van mijn game. Hierbij heb ik gelet op:

- de **tilability** van de tiles (ze sluiten goed op elkaar aan zonder zichtbare randen),
- een **coherente stijl** met de andere elementen van mijn spel,
- een **eenvoudige maar leesbare vormgeving**, zodat het toegankelijk blijft voor mijn doelgroep (11–15 jaar).

Door deze keuze heb ik me bewust **losgemaakt van bestaande templates of asset packs**. Mijn bedoeling was om **een eigen visuele identiteit** te creëren die past bij de rest van het spel, zonder afhankelijk te zijn van externe bronnen.

Deze toevoeging toont aan dat ik niet alleen technisch heb bijgedragen aan het spel, maar ook **grafisch meegebouwd heb aan de spelwereld**, wat een belangrijke meerwaarde is binnen het kader van mijn tweede zit.



18. Aanpassingen aan mijn magazine – visuele samenhang met het spel

Tijdens mijn tweede zit heb ik niet enkel gewerkt aan de game zelf, maar ook aan mijn magazine. Ik vond het belangrijk dat dit magazine dezelfde **stijl en sfeer** uitstraalt als mijn educatieve game **Save The Planet**, zodat beide onderdelen samen een coherent geheel vormen.

Daarom heb ik de volgende aanpassingen doorgevoerd:

- **Pixelart-stijl toegepast** op de vormgeving, zodat het magazine visueel aansluit bij de retrolook van mijn spel.
- **Titels typografie aangepast** naar een pixelachtige stijl, wat zorgt voor consistentie met de in-game UI.
- **Illustraties vervangen** in pixelart, zodat ze dezelfde eenvoud en speelse toon uitstralen als de spelwereld.
- **Nieuwe mock-ups toegevoegd** waarin de geüpdatete versie van mijn spel (zoals het hoofdmenu, quiz, eco-score en NPC's) duidelijk zichtbaar is.

Dankzij deze aanpassingen voelt het magazine niet losgekoppeld aan, maar als een **verlengstuk van het spel**. Het benadrukt de samenhang tussen spel, documentatie en einddoelstelling.

Mijn onderzoeksvraag:

Hoe kunnen we een educatief spel ontwerpen dat jongeren bewust maakt van klimaatproblemen en hen aanzet tot duurzamer gedrag?

Hoe mijn game dit beantwoordt:

- In *Save The Planet* voeren jongeren drie concrete acties uit: bomen planten, afval opruimen, en energie besparen.
- Bij elke actie verandert de spelwereld én dalen de live-statistieken bovenaan het scherm.
- NPC's geven weetjes in een verhalende vorm, gekoppeld aan de thema's.
- Op het einde krijgen spelers een eco-score, gebaseerd op hun keuzes.
- Alles is visueel, interactief en afgestemd op hun leefwereld.

Dus het spel maakt duidelijk: **jouw keuzes hebben impact**, en dat is precies wat de onderzoeksvraag onderzoekt.

Met dit document toon ik aan dat ik tijdens mijn tweede zit actief en zelfstandig gewerkt heb aan mijn spel. Ik heb gekozen om verschillende onderdelen volledig zelf te ontwerpen, te programmeren en te verbeteren. Deze aanpak toont niet alleen mijn technische groei, maar ook mijn persoonlijke betrokkenheid bij het project. Ik hoop dat dit duidelijk aantoont dat mijn eigen inzet tijdens de herwerking van het spel wel degelijk aanwezig is.